Official Team Role Breakdown & Deliverable Expectations

Project: Airport Operations Coordination System (AOCS)
Document Version: 1.0
Date: 2026-07-23

🗄️ 1. Krishna Solanki — Database & System Integration

- **Domain**: Data Architecture, ORM, Database Migrations, & Full-Stack API Integration
- **Primary Tech**: PostgreSQL, Spring Data JPA, Flyway, Axios/Fetch API Wiring

Core Deliverables:

1. **Database Schema & Migrations (`schema.sql` & Flyway)**:
   - Maintain the PostgreSQL database structure (`users` , `roles` , `flights` , `gates` , `tasks` , etc.).
   - Write Flyway versioned migration scripts (e.g., `V1__init_schema.sql` , `V2__seed_sample_data.sql`) so teammates' local databases update automatically upon pulling code.
2. **Spring Data JPA Entities & Repositories**:
   - Create Java Entity classes (e.g., `@Entity Flight` , `@Entity TurnaroundTask`) mapping tables to Spring Boot.
   - Write repository interfaces (`FlightRepository` , `TaskRepository`) for database queries.
3. **Frontend-to-Backend Connection (Integration)**:
   - Wire the React frontend screens (built by Anuvrat) to the Spring Boot REST APIs (built by Anay).
   - Handle CORS headers, HTTP request/response payloads, loading states, and error handling during data transfers.

4. **Viva Expectation**:
    - Explain database table relationships (Foreign Keys, Indexes), Spring Boot to PostgreSQL connectivity, and data flow to the React UI.

📝 2. Chaitanya Tikku — Documentation, UML & QA Testing

- **Domain**: Academic Submission Documents, Diagrams, & Quality Assurance
- **Primary Tools**: Draw.io / Mermaid, Postman, MS Word / Markdown, JUnit / Jest

Core Deliverables:

1. **Software Requirement Specification (SRS Document)**:
    - Write and format the official SRS document (10 marks evaluation item) detailing project scope, 15+ functional requirements, and non-functional constraints.
2. **UML Diagrams**:
    - **Use Case Diagram**: Showing how Administrators, Supervisors, and Ground Crew interact with the system.
    - **Class Diagram**: Object structure matching the Java backend entities.
    - **Sequence Diagrams**: Step-by-step turnaround workflow (*Aircraft Lands ➔ Gate Assigned ➔ Tasks Triggered ➔ Flight Departed*).
3. **API & QA Testing**:
    - Test backend APIs using **Postman** (creating a Postman Collection of all GET, POST, PUT, DELETE requests).
    - Verify input validations (e.g., preventing invalid gate assignments, wrong timestamps) and log bug reports.
4. **User Manual & Final Presentation (PPT)**:
    - Create the step-by-step User Manual guide for operating the AOCS software.
    - Design the final PowerPoint presentation slides for evaluation day.
5. **Viva Expectation**:
    - Explain system architecture diagrams, use cases, test cases, and project requirements to the faculty guide.

🎨 3. Anuvrat Tripathi — Frontend UI/UX

- **Domain**: Web Interface, User Experience, & Screen Components
- **Primary Tech**: React, TypeScript, Material UI (MUI), HTML5, CSS3

Core Deliverables:

1. **15+ Web Screens**:
    - Build UI screens: Dashboard Overview, Flight Schedule Grid, Gate Allocation Map, Task Cards, Department View, User Login/Profile, and Reports Page.
2. **Reusable UI Component Library**:
    - Design Material UI components: Status Chips (e.g., Green `READY`, Red `DELAYED`), Flight Progress Bars, Priority Badges (*Normal, High, Critical*), and Alert Snackbars.
3. **Client-Side Form Validation**:
    - Build interactive forms for task updates, gate changes, and login with instant validation feedback.
4. **Viva Expectation**:
    - Demonstrate the user interface, component structure, responsiveness, and state management in React.

⚙️ 4. Anay Modi — Backend API & Business Logic

- **Domain**: REST API Services, Turnaround Rules, & Security
- **Primary Tech**: Java 17/21, Spring Boot, Spring Security (RBAC)

Core Deliverables:

1. **REST Controller Endpoints**:
    - Build backend endpoints: `/api/auth`, `/api/flights`, `/api/gates`, `/api/tasks`, `/api/notifications`, `/api/reports`.
2. **Business Logic & Rules**:
    - Program turnaround rules (e.g., an aircraft cannot board until cabin cleaning and

refueling tasks are marked `COMPLETED`).

3. **Role-Based Security (RBAC)**:
    - Restrict access so ground crew members can only update their department's tasks, while supervisors can manage all flights and gates.
4. **Delay Calculation Engine**:
    - Logic that monitors planned vs. actual timestamps and automatically flags delayed tasks.
5. **Viva Expectation**:
    - Explain the Java service architecture, Spring Controllers, business logic rules, and security filters.

📊 Summary Matrix for Evaluation

Team Member	Primary Technical Responsibility	Key Output Files
Krishna Solanki	Database, JPA Entities, & API Integration	`schema.sql`, JPA Entities, API Connection Hooks
Chaitanya Tikku	SRS, UML Diagrams, Test Cases, & PPT	`SRS.pdf`, UML Diagrams, Postman Tests, Presentation
Anuvrat Tripathi	React UI Screens & MUI Components	`Dashboard.tsx`, `FlightList.tsx`, UI Components
Anay Modi	Spring Boot Controllers & Business Logic	`FlightController.java`, `TaskService.java`, Security Config